

# WEBRUM-08: Dashboards and Alerting

---

**Series:** WEBRUM — Web Real User Monitoring | **Notebook:** 8 of 8 | **Created:** March 2026 | **Last Updated:** 04/04/2026

## Overview

---

An effective RUM monitoring strategy requires both visibility (dashboards) and proactive detection (alerting). Dashboards provide at-a-glance health views for different audiences — executives care about business impact and Apdex trends, while operations teams need real-time error rates and performance breakdowns.

This notebook covers building executive and operational RUM dashboards, calculating Apdex scores via DQL, creating RUM-based alerts for error rate spikes and performance degradation, and combining RUM with synthetic monitoring for a complete UX picture.

---

## Table of Contents

---

- [1. Executive RUM Dashboard KPIs](#)
  - [2. Apdex Score Calculation](#)
  - [3. Operational RUM Dashboard](#)
  - [4. RUM Alerting Strategies](#)
  - [5. Error Rate Alerts](#)
  - [6. Performance Degradation Alerts](#)
  - [7. RUM + Synthetic Combined View](#)
  - [8. Summary and Series Recap](#)
-

# Prerequisites

| Requirement           | Details                                                                                             |
|-----------------------|-----------------------------------------------------------------------------------------------------|
| Dynatrace Environment | SaaS with Grail enabled                                                                             |
| RUM Enabled           | Web applications with at least 7 days of data                                                       |
| Permissions           | <code>storage:events:read</code> , <code>storage:metrics:read</code> , dashboard create permissions |
| Previous Notebooks    | All prior WEBRUM notebooks (01-07)                                                                  |

## 1. Executive RUM Dashboard KPIs

Executive dashboards should be simple, business-focused, and actionable. Key metrics:

| KPI                  | What It Tells Executives           | Target              |
|----------------------|------------------------------------|---------------------|
| Apdex score          | Overall user satisfaction (0-1)    | > 0.85              |
| Session count        | Traffic volume and trends          | Depends on business |
| Error rate %         | Percentage of sessions with errors | < 2%                |
| Bounce rate %        | Users leaving after one page       | < 40%               |
| CWV pass rate        | % of page loads meeting all 3 CWV  | > 75%               |
| Avg session duration | User engagement level              | Varies by app type  |

```
// Executive KPI summary – single-query dashboard tile
fetch user.sessions, from:-24h
| filter userType == "REAL_USER"
| summarize
    total_sessions = count(),
    error_sessions = countIf(totalErrorCount > 0),
    bounce_sessions = countIf(userActionCount == 1),
    avg_actions = avg(userActionCount),
    avg_duration_min = avg(toDouble(duration) / 60000000000.0),
    by:{application}
| fieldsAdd error_rate_pct = round(toDouble(error_sessions) /
toDouble(total_sessions) * 100.0, decimals: 1),
    bounce_rate_pct = round(toDouble(bounce_sessions) /
toDouble(total_sessions) * 100.0, decimals: 1),
    avg_duration_min = round(avg_duration_min, decimals: 1)
| sort total_sessions desc
```

```
// Session volume trend – 7-day daily session counts
fetch user.sessions, from:-7d
| filter userType == "REAL_USER"
| makeTimeseries session_count = count(), interval:1d, by:{application}
```

## 2. Apdex Score Calculation

The **Application Performance Index (Apdex)** is an open standard for measuring user satisfaction. It classifies every user action into three buckets based on a threshold T:

| Classification    | Duration                      | Description                             |
|-------------------|-------------------------------|-----------------------------------------|
| <b>Satisfied</b>  | $\leq T$                      | Action completed within acceptable time |
| <b>Tolerating</b> | $T < \text{duration} \leq 4T$ | Action was slow but user waited         |
| <b>Frustrated</b> | $> 4T$                        | Action was too slow or failed           |

**Apdex formula:**

```
Apdex = (Satisfied + (Tolerating / 2)) / Total
```

The threshold T is configurable per application. Common defaults:

| Action Type  | Default T   |
|--------------|-------------|
| Page load    | 3 seconds   |
| XHR action   | 2.5 seconds |
| Route change | 2.5 seconds |

```
// Apex calculation with T = 3 seconds for page loads
fetch user.events, from:-24h
| filter action.type == "Load"
| fieldsAdd duration_sec = toDouble(duration) / 1000000000.0
| summarize total = count(),
    satisfied = countIf(duration_sec <= 3),
    tolerating = countIf(duration_sec > 3 and duration_sec <= 12),
    frustrated = countIf(duration_sec > 12),
    by:{application}
| fieldsAdd apdex = round((toDouble(satisfied) + toDouble(tolerating) / 2.0)
/ toDouble(total), decimals: 3)
| sort apdex asc
```

```
// Apex trend over 7 days – daily Apex score
fetch user.events, from:-7d
| filter action.type == "Load"
| fieldsAdd duration_sec = toDouble(duration) / 1000000000.0
| fieldsAdd is_satisfied = if(duration_sec <= 3, 1.0, else: 0.0),
    is_tolerating = if(duration_sec > 3 and duration_sec <= 12, 0.5,
else: 0.0)
| makeTimeseries
    total_actions = count(),
    satisfied_score = sum(is_satisfied),
    tolerating_score = sum(is_tolerating),
    interval:1d
```

```
// Apdex by page – which pages have the worst user satisfaction?
fetch user.events, from:-24h
| filter action.type == "Load"
| fieldsAdd duration_sec = toDouble(duration) / 1000000000.0
| summarize total = count(),
    satisfied = countIf(duration_sec <= 3),
    tolerating = countIf(duration_sec > 3 and duration_sec <= 12),
    by:{action.name}
| filter total > 20
| fieldsAdd apdex = round((toDouble(satisfied) + toDouble(tolerating) / 2.0)
/ toDouble(total), decimals: 3)
| sort apdex asc
| limit 10
```

## Interpreting Apdex Scores

| Apdex Range | Rating       | Action                       |
|-------------|--------------|------------------------------|
| 0.94 - 1.00 | Excellent    | No action needed             |
| 0.85 - 0.93 | Good         | Monitor for changes          |
| 0.70 - 0.84 | Fair         | Investigate slow pages       |
| 0.50 - 0.69 | Poor         | Prioritize optimization      |
| < 0.50      | Unacceptable | Immediate attention required |

## 3. Operational RUM Dashboard

Operations teams need real-time visibility into errors, performance anomalies, and traffic patterns.

```
// Real-time error rate – errors per 15 minutes over the last 6 hours
fetch user.events, from:-6h
| filter type == "Error"
| makeTimeseries error_count = count(), interval:15m, by:{error.type}
```

```
// Active error summary – current top errors in the last hour
fetch user.events, from:-1h
| filter type == "Error"
| summarize error_count = count(),
    affected_sessions = countDistinct(sessionId),
    by:{error.message, error.type, application}
| sort affected_sessions desc
| limit 10
```

```
// Performance SLA – percentage of page loads under 3 seconds
fetch user.events, from:-1h
| filter action.type == "Load"
| fieldsAdd duration_sec = toDouble(duration) / 1000000000.0
| summarize total = count(),
    under_3s = countIf(duration_sec <= 3),
    by:{application}
| fieldsAdd sla_pct = round(toDouble(under_3s) / toDouble(total) * 100.0,
    decimals: 1)
| sort sla_pct asc
```

## 4. RUM Alerting Strategies

Effective RUM alerting requires the right balance — too sensitive triggers alert fatigue, too loose misses real issues.

### Recommended Alert Categories

| Category                       | What to Alert On                  | Threshold Example                      |
|--------------------------------|-----------------------------------|----------------------------------------|
| <b>Error rate spike</b>        | Sudden increase in JS error rate  | > 5% of sessions with errors           |
| <b>Apdex drop</b>              | User satisfaction below threshold | Apdex < 0.7 for 15 minutes             |
| <b>Performance degradation</b> | p75 page load exceeds baseline    | p75 > 5s for 15 minutes                |
| <b>Traffic anomaly</b>         | Unexpected drop in session volume | < 50% of previous day's hourly average |

| Category              | What to Alert On                   | Threshold Example           |
|-----------------------|------------------------------------|-----------------------------|
| <b>CWV regression</b> | Core Web Vitals crossing into Poor | LCP p75 > 4s for 30 minutes |

## Alert Configuration via Metrics Events

Dynatrace metric events allow alerting on RUM metrics. Configure via:

**Settings > Anomaly detection > Metric events > Add metric event**

| Setting                  | Recommended Value                                              |
|--------------------------|----------------------------------------------------------------|
| <b>Metric</b>            | RUM metric (e.g., <code>builtin:apps.web.action.apdex</code> ) |
| <b>Evaluation window</b> | 15 minutes (balances speed and noise)                          |
| <b>Sliding window</b>    | 5-minute intervals                                             |
| <b>Threshold type</b>    | Static or auto-adaptive baseline                               |

## 5. Error Rate Alerts

The following queries provide the data foundation for error rate alerting. Use these patterns to detect when error rates exceed acceptable thresholds.

```
// Error rate per 15-minute window – alert threshold data
fetch user.sessions, from:-6h
| filter userType == "REAL_USER"
| fieldsAdd has_error = if(totalErrorCount > 0, 1.0, else: 0.0)
| makeTimeseries
  total_sessions = count(),
  error_sessions = sum(has_error),
  interval:15m,
  by:{application}
```

```
// New errors – errors first seen in the last getHour(potential deployment issue)
fetch user.events, from:-1h
| filter type == "Error"
| summarize first_seen = min(timestamp),
    error_count = count(),
    affected_sessions = countDistinct(sessionId),
    by:{error.message, application}
| sort first_seen desc
| limit 10
```

## 6. Performance Degradation Alerts

Detect when page load performance degrades beyond acceptable thresholds.

```
// p75 page load duration per 15-minute window – performance alert data
fetch user.events, from:-6h
| filter action.type == "Load"
| fieldsAdd duration_ms = toDouble(duration) / 1000000.0
| makeTimeseries p75_duration = percentile(duration_ms, 75), interval:15m,
by:{application}
```

```
// Apdex per 15-minute window – satisfaction alert data
fetch user.events, from:-6h
| filter action.type == "Load"
| fieldsAdd duration_sec = toDouble(duration) / 1000000000.0
| fieldsAdd apdex_score = if(duration_sec <= 3, 1.0,
    else: if(duration_sec <= 12, 0.5,
    else: 0.0))
| makeTimeseries avg_apdex = avg(apdex_score), interval:15m, by:{application}
```

**Tip:** For production alerting, use Dynatrace Intelligence anomaly detection rather than static thresholds. Dynatrace Intelligence automatically baselines normal behavior and detects deviations, reducing false positives from seasonal traffic patterns.



## 7. RUM + Synthetic Combined View

RUM and synthetic monitoring complement each other. Combining them provides a complete UX monitoring picture:

| Aspect                 | RUM Covers                       | Synthetic Covers               |
|------------------------|----------------------------------|--------------------------------|
| <b>Availability</b>    | Only when users visit            | 24/7 scheduled monitoring      |
| <b>Baseline</b>        | Real-world conditions (variable) | Clean-room conditions (stable) |
| <b>Coverage</b>        | Pages users visit                | All critical paths (scripted)  |
| <b>Error detection</b> | Errors users hit                 | Errors in scripted flows       |
| <b>Geographic</b>      | Where users are                  | From configured locations      |

### Side-by-Side Comparison

Use `append` to compare RUM and synthetic performance for the same application:

```
// RUM session summary – real user experience
fetch user.sessions, from:-24h
| filter userType == "REAL_USER"
| summarize rum_sessions = count(),
    rum_error_sessions = countIf(totalErrorCount > 0),
    rum_avg_actions = avg(userActionCount),
    by:{application}
| fieldsAdd monitoring_type = "RUM",
    error_rate_pct = round(toDouble(rum_error_sessions) /
toDouble(rum_sessions) * 100.0, decimals: 1)
```

```
// Synthetic vs RUM – compare session types for the same app
fetch user.sessions, from:-24h
| summarize session_count = count(),
    avg_duration_sec = avg(toDouble(duration) / 1000000000.0),
    error_sessions = countIf(totalErrorCount > 0),
    by:{application, userType}
| filter userType == "REAL_USER" or userType == "SYNTHETIC"
| fieldsAdd error_rate_pct = round(toDouble(error_sessions) /
toDouble(session_count) * 100.0, decimals: 1)
| sort application asc, userType asc
```

## 8. Summary and Series Recap

---

In this notebook, we covered:

- **Executive KPIs** — Session count, error rate, bounce rate, Apdex
- **Apdex calculation** — Satisfied/tolerating/frustrated classification via DQL
- **Operational dashboard** — Real-time error rates, SLA tracking, active errors
- **Alerting strategies** — Error rate spikes, Apdex drops, performance degradation
- **RUM + Synthetic** — Combining both monitoring types for complete coverage

### WEBRUM Series Recap

| Notebook | Topic                 | Key Takeaway                                         |
|----------|-----------------------|------------------------------------------------------|
| 01       | RUM Fundamentals      | JavaScript agent, data model, basic queries          |
| 02       | SPA Instrumentation   | Route changes, framework tips, XHR monitoring        |
| 03       | Core Web Vitals       | LCP, INP, CLS measurement and scoring                |
| 04       | Session Analysis      | Segmentation, journeys, conversions, bounce rates    |
| 05       | Error Analysis        | Error impact, rage clicks, error-session correlation |
| 06       | Performance Analysis  | Waterfall, TTFB, slow pages, geographic performance  |
| 07       | Session Replay        | Privacy, masking, finding and correlating replays    |
| 08       | Dashboards & Alerting | Apdex, KPI dashboards, alerting strategies           |

### References

- [Apdex Standard](#)
  - [Dynatrace RUM Dashboards](#)
  - [Metric Events for Alerting](#)
-

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*